

FACULTY OF SCIENCE, ENGINEERING AND ENVIRONMENT



ASSESMENT REPORT (NATURAL LANGUAGE PROCESSING)

TensorFlow-based Text Classification Starting with Raw Text and
Using the Embedding Space

SUBMITTED BY

MOYOSOOREOLUWA OKEYODE (@00798529)

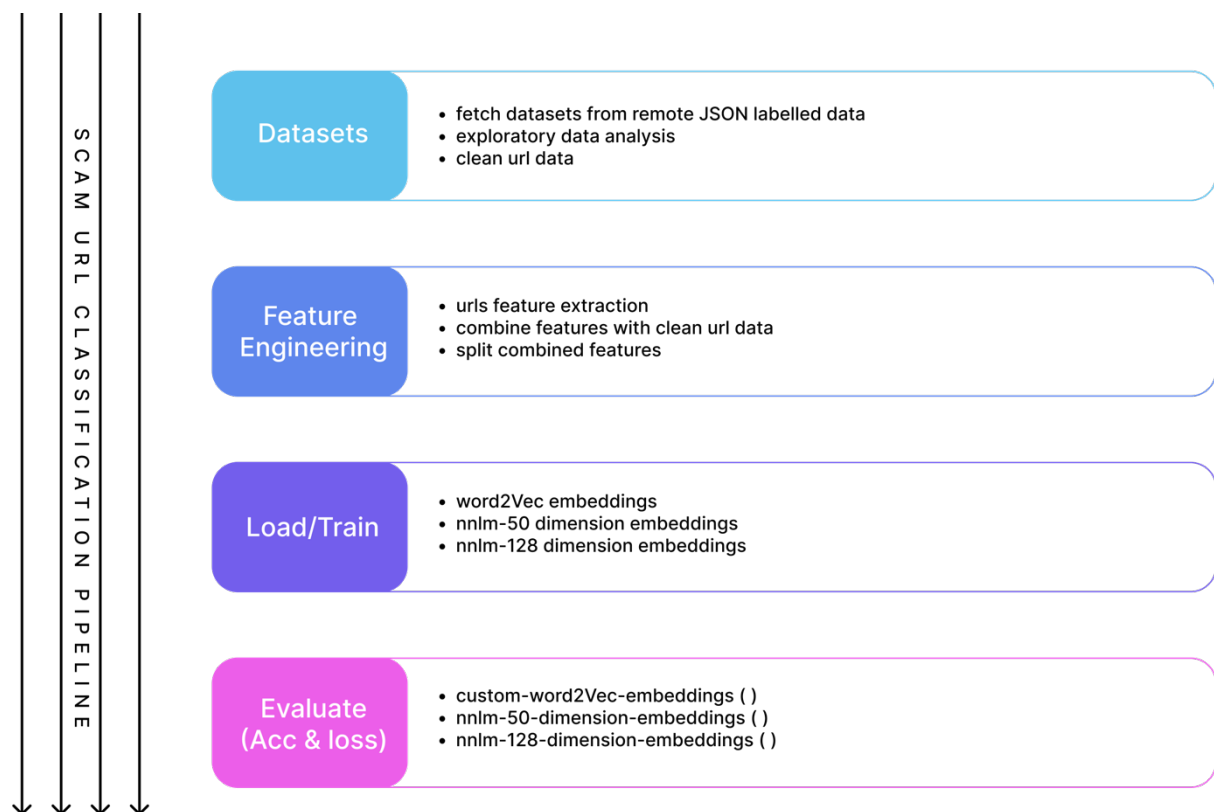
DATE

21-03-2025

1. INTRODUCTION

The rapid growth of cryptocurrency adoption has been accompanied by a surge in sophisticated online scams, particularly through fraudulent URLs. These malicious links often mimic legitimate cryptocurrency exchanges, wallet services, or investment platforms to deceive users into revealing sensitive information or transferring funds. The cryptocurrency scam losses about \$3.3 billion in 2023, with phishing URLs being one of the primary attack vectors. This features the critical need for robust automated detection systems that can identify scam URLs.

Natural Language Processing (NLP) techniques have shown remarkable success in text classification tasks, making them well-suited for analyzing URL structures and content. However, the effectiveness of these techniques depends heavily on the choice of text representation methods. Traditional approaches like bag-of-words or TF-IDF lack the semantic understanding required to detect sophisticated scams, while modern embedding techniques like Word2Vec, NNLM, GloVe and BERT and can capture deeper linguistic patterns and contextual relationships in URLs.



2. DATASETS

Dataset Sources

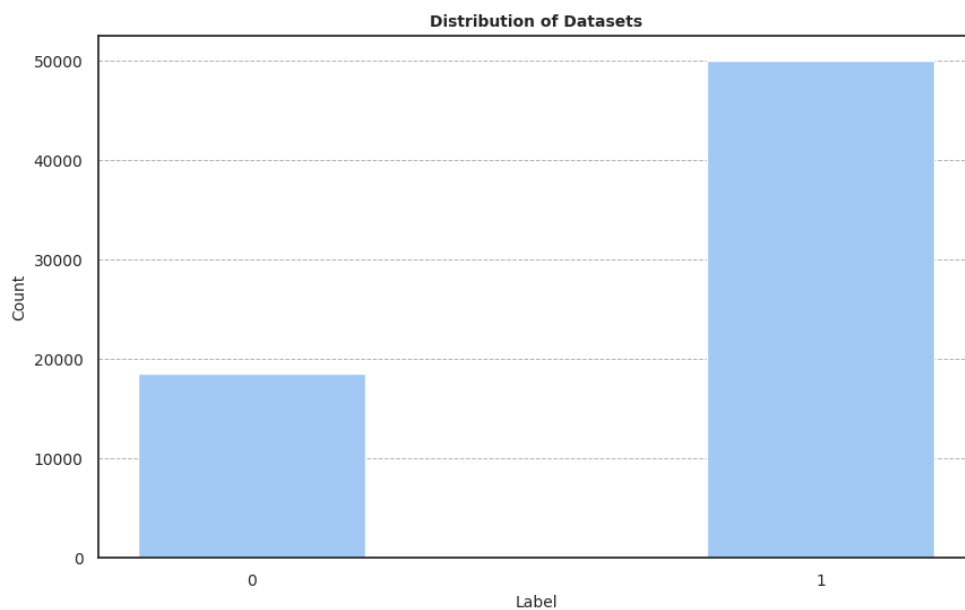
The chosen dataset is a collection of crypto-related URLs, labeled as either "legitimate" or "scam". The Legitimate URLs were generated from Coingecko's free API and Scam URLs generated from curated Ethereum Phishing Tank.

Data Exploration

The dataset used for this task comprises URLs labeled as either legitimate or scam, aimed at supporting a binary classification task. An in-depth exploratory data analysis (EDA) was conducted to uncover patterns, anomalies, and informative features that can contribute to model performance.

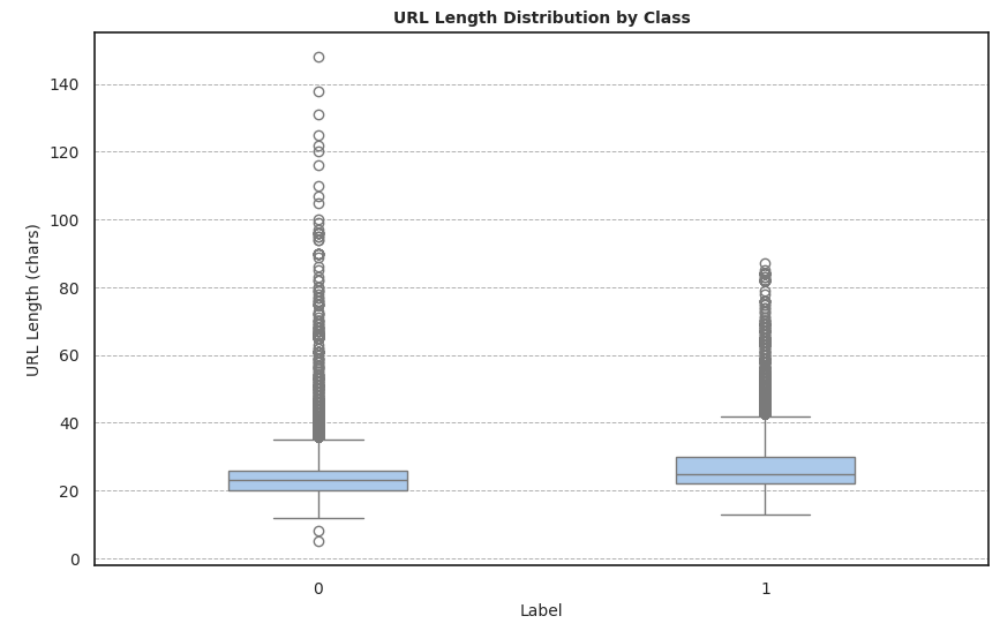
1. Class Distribution

A preliminary analysis of the label distribution revealed 18,595 Legitimate URLs and 50,000 Scam URLs. Given the potential impact of class imbalance on model training, mitigation strategies such as resampling or the use of class weights will be considered for the task.



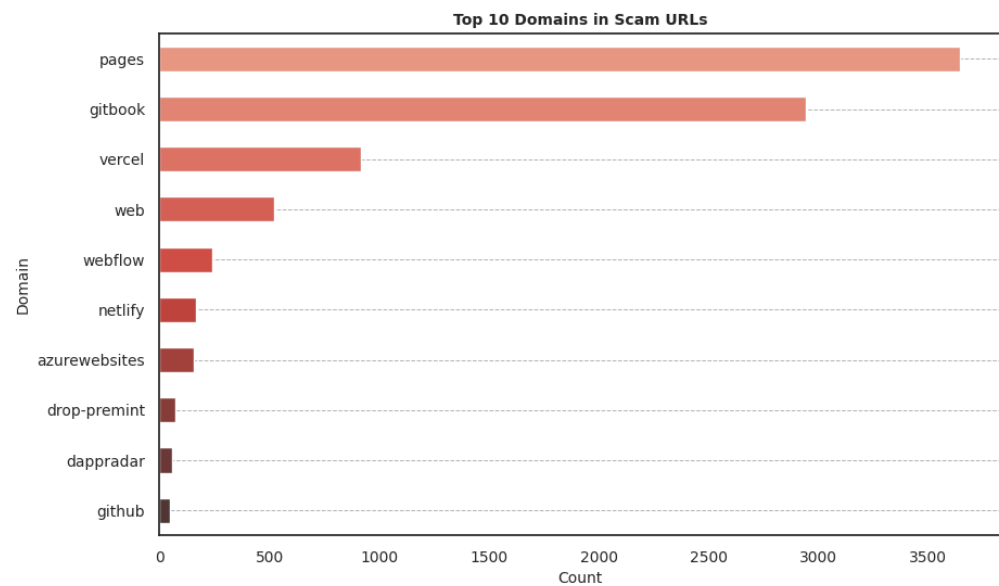
2. URL Structure and Length

Scam URLs often exhibit distinct structural patterns when compared to legitimate ones. The length distribution indicated that scam URLs tend to be longer on average, often containing random strings or suspicious words designed to mimic trustworthy links.



3. Domain and Subdomain Patterns

Analysis of domain and subdomain components showed that scam URLs often originate from low-reputation or uncommon top-level domains (TLDs). Many also use subdomains resembling well-known brands, which may mislead users into believing the URLs are authentic.



The visualization highlights the top 10 domains most used in scam URLs, revealing that platforms like pages, gitbook, and vercel are the most frequently abused, with pages alone appearing in over 3,600 scam links. These domains, often associated with free or developer-friendly hosting services, are likely targeted by scammers for their accessibility and ease of deployment.

Data Preprocessing

Preprocess raw URL data to engineer meaningful features for deep learning models (e.g., LSTM with embeddings).

1. Data Cleaning

- **Whitespace Removal & Lowercasing:** Strips extra spaces and converts the URL to lowercase for uniformity.
- **Scheme Removal:** Removal of "http://", "https://", and "www." prefixes to focus on the URL structure.
- **TLD Removal:** Removes top-level domains (e.g., .com, .org, .xyz) to normalize URL structure.
- **Tokenization:** Splits the URL into meaningful tokens based on common separators.

2. URL Features Extraction

- **Suspicious Keyword Detection:** Check if the URL contains keywords like "free", "bonus", "crypto", etc., which are often associated with scams.
- **Shortened URL Check:** Identify if the URL uses a URL shortening service (e.g., bit.ly, goo.gl).
- **URL Length:** Capture the total length of the URL.
- **Special Characters:** Check for the presence of special characters (e.g., @, #, &).
- **IP Address Detection:** Identify if the URL contains an IP address.
- **Suspicious Keyword Flag:** Flag the URL if any suspicious keywords are found.
- **Typo Squatting Check:** Compare the URL domain with legitimate domains to detect potential typo-squatting.
- **TLD (Top-Level Domain):** Extract and store the TLD (e.g., .com, .xyz).
- **Domain Age:** Calculate the domain age in days.

Processing each URL to extract components like the domain, TLD, and netloc. It evaluates characteristics such as suspicious keywords, URL length, special characters, and domain age.

Dataset Splitting

After the features are combined, the dataset is split into two parts: the feature set (X) and the target labels (y). The features are separated from the labels, which indicate whether a URL is a scam or legitimate. The dataset is then divided into training and testing subsets using an 80-20 split before combining with other features, ensuring that there is no **data leakage**.

```
# Separate features and labels
X = datasets_df['cleaned_url']
y = datasets_df['label']

# Split into train/test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Python

Features Combination

The features which are generated are combined with the embeddings of the dataset that has been split, creating a comprehensive dataset that includes both the embeddings and the corresponding features. This combined dataset provides the necessary input for training the model.

```
# Combine Word2Vec with features
X_train_w2v_combined = np.concatenate([X_train_w2v_embed, X_train_features_scaled], axis=1)
X_test_w2v_combined = np.concatenate([X_test_w2v_embed, X_test_features_scaled], axis=1)

# Combine NNLM-50 with features
X_train_50_combined = np.concatenate([X_train_embed_50, X_train_features_scaled], axis=1)
X_test_50_combined = np.concatenate([X_test_embed_50, X_test_features_scaled], axis=1)

# Combine NNLM-128 with features
X_train_128_combined = np.concatenate([X_train_embed_128, X_train_features_scaled], axis=1)
X_test_128_combined = np.concatenate([X_test_embed_128, X_test_features_scaled], axis=1)
```

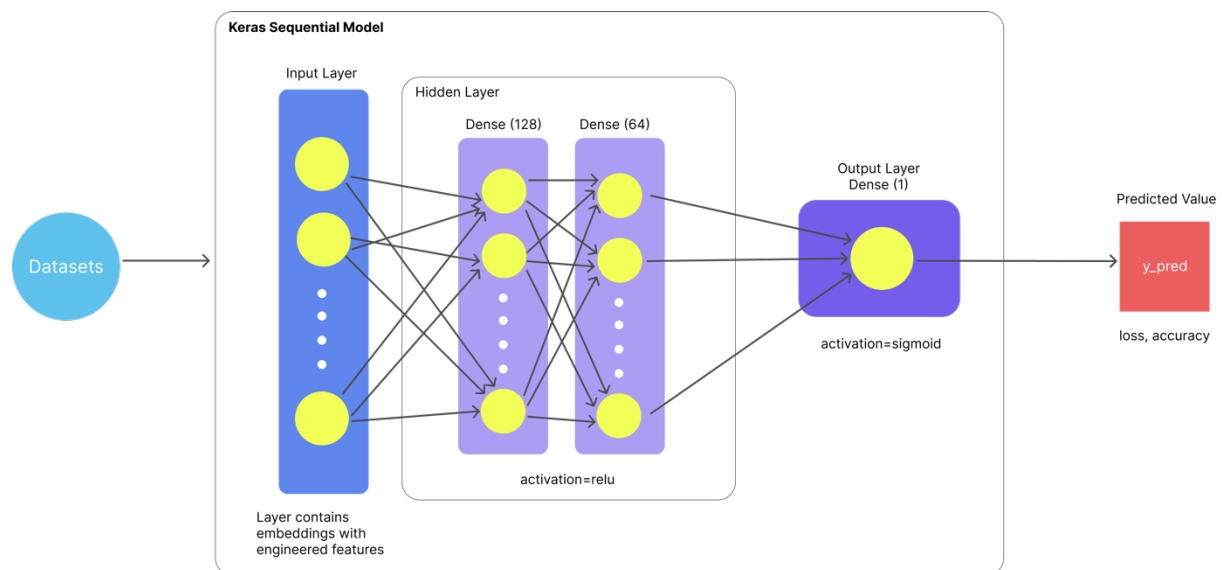
Python

3. SYSTEM IMPLEMENTATION ON GOOGLE COLAB

3.1 Model Architecture

The system uses a sequential neural network built with TensorFlow/Keras, designed to classify input samples into two classes, Legitimate or Scam.

Our Neural network model is a sequence of layers as below:



- **Input Layer:** Accepts feature vectors (e.g., embedded urls and engineered numeric features). The input shape matches the number of input features.
- **Hidden Dense Layers:**
 - **First Layer:** A fully connected layer with 128 neurons and relu activation to introduce non-linearity and capture complex patterns.
 - **Second Layer:** Another dense layer with 64 neurons and relu activation to refine feature transformation.
- **Output Layer:**
 - For the binary classification, a single neuron with sigmoid activation is used.
 - The model is trained using binary cross-entropy (for 2 classes).

3.2 Libraries Used

The system was implemented using the following platforms and libraries:

Library	Purpose	Reason for Selection
nlTK	Natural Language Processing (tokenization, text processing).	Preprocess URL/text data (e.g., tokenization for feature extraction).
re	Regular expressions for pattern matching in strings.	Clean URLs, extract patterns (e.g., special characters, suspicious keywords).
whois	Fetch domain registration details (creation date, registrar).	Extract WHOIS features (e.g., domain age, owner) for phishing detection.
tldextract	Extract TLD (Top-Level Domain), domain, and subdomain from URLs.	Better than urlparse for accurate domain segmentation (e.g., google.co.uk).
urllib.parse.urlparse	Parse URLs into components (scheme, netloc, path).	Analyze URL structure (e.g., detect suspicious paths/parameters).
datetime	Handle date/time operations.	Calculate domain age from WHOIS data.
functools.lru_cache	Cache function results to avoid redundant computations.	Optimize WHOIS lookups or repetitive feature extraction.
rapidfuzz	Fuzzy string matching (e.g., similarity scores between strings).	Compare URLs to known phishing templates (faster than fuzzywuzzy).
collections.Counter	Count hashable objects (e.g., word frequencies).	Analyze token/character distributions in URLs.

Machine Learning & NLP Libraries

Library	Purpose	Reason for Selection
gensim	Topic modeling, word embeddings (e.g., Word2Vec).	Generate semantic features from URL/text (if NLP-based detection is used).
sklearn	ML pipelines, preprocessing, and models (MLPClassifier).	Standard library for ML workflows (train/test splits, scaling, evaluation metrics).
tensorflow	Deep learning (neural networks).	Build/train complex models (e.g., CNNs for URL analysis).

tensorflow_hub	Pretrained models (e.g., TF-Hub embeddings).	Leverage transfer learning (e.g., Universal Sentence Encoder for URL text).
keras	High-level NN API (layers, models).	Simplify model architecture design (e.g., Sequential models).

Data Visualization and Evaluation

Library	Purpose	Reason for Selection
seaborn	Statistical visualizations (heatmaps, distributions).	Enhance plots with minimal code (e.g., confusion matrix heatmaps).
matplotlib.pyplot	Custom plotting (e.g., loss curves, bar charts).	Fine-tune visualizations beyond seaborn defaults.
sklearn.metrics	Model evaluation (classification_report, confusion_matrix).	Standard metrics for classification performance.
ConfusionMatrixDisplay	Visualize confusion matrices.	Integrated with skl

3.3 Embeddings Techniques

The task focuses on evaluating three distinct embedding spaces: **Word2Vec**, **nnlm-50**, and **nnlm-128**. These embeddings will be used to represent textual data for the purpose of building a robust detection model. The key objective is to analyze and compare the performance of each embedding in capturing semantic relationships within the URLs and its impact on the overall accuracy of the model.

1. Word2Vec Embeddings

Word2Vec is a widely used word embedding technique that learns vector representations of words based on their contextual patterns in large text corpora. It operates using either the Continuous Bag-of-Words (CBOW) or Skip-gram architectures, efficiently capturing semantic relationships between words.

- Helps identify suspicious keywords (e.g., "bitcoin," "free," "wallet") by their vector similarities.
- Useful for detecting phrasing patterns commonly found in scam URLs (e.g., "claim-your-reward," "verify-account").

2. NNLM – 50 Embeddings

NNLM is a neural language model that learns word embeddings by predicting the next word in a sequence. The 50-dimensional variant produces compact embeddings while retaining meaningful linguistic features.

- Captures sequential dependencies in URLs (e.g., "login-secure-crypto.com" vs. "login-scam-site.com").
- Better at handling out-of-vocabulary (OOV) words compared to Word2Vec.

3. NNLM – 128 Embeddings

An extended version of NNLM with 128-dimensional embeddings, offering richer semantic representations by capturing more nuanced linguistic patterns.

- More effective in detecting complex scam patterns due to higher-dimensional feature space.
- Better generalization for diverse scam URL structures.

3.3 Model Compilation

The following parameters will be used in compiling the model.

Optimizer: ‘Adam’ Adaptive Moment Estimation. It’s a popular and efficient optimization algorithm that adjusts learning rates automatically during training.

Loss: ‘Binary Crossentropy’ This is the loss function used for binary classification (e.g., scam vs. legit). It measures how far the predicted probabilities are from the actual labels (0 or 1). Lower values mean better predictions.

Metrics: ‘Accuracy’ this tells the model to track accuracy during training and evaluation. Accuracy is the proportion of correct predictions (e.g., predicting scam URLs as scams).

```

def train_model(X_train, y_train, X_test, y_test):
    model = models.Sequential([
        layers.InputLayer(shape=(X_train.shape[1],)),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(1, activation='sigmoid')
    ])

    # Compile the model
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    # Model summary
    model.summary()

    # Train the model
    history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))

    # evaluate the model
    loss, accuracy = model.evaluate(X_test, y_test)
    print(f"Test Loss: {loss:.4f}, Test Accuracy: {accuracy:.4f}")

    # predict on the test set
    y_pred = (model.predict(X_test) > 0.5).astype(int)

    # create confusion matrix
    confusion = confusion_matrix(y_test, y_pred)

    # print classification report
    print("\n=== Classification Report ===")
    print(classification_report(y_test, y_pred))

    # display the confusion matrix
    disp = ConfusionMatrixDisplay(confusion_matrix=confusion, display_labels=map.keys())
    disp.plot(cmap=plt.cm.Reds)
    plt.title("Confusion Matrix")
    plt.show()

    return history

```

function to train model

4. RESULT ANALYSIS

4.1 Model Evaluation

Three different embedding techniques Word2Vec, NNLM-50, and NNLM-128 paired with a Multi-Layer Perceptron (MLP) classifier trained over 20 Epochs to assess their effectiveness in identifying scam cryptocurrency URLs with metrics reported for accuracy and loss.

Word2Vec Embeddings The model using Word2Vec embeddings achieved a train accuracy of 87.99% test accuracy of 87.46%, with strong performance in identifying class 1 (precision: 0.90, recall: 0.93) but slightly weaker results for class 0 (precision: 0.79, recall: 0.74). Training and validation accuracy improved steadily, showing good learning behavior and generalization, though the model missed more negatives than positives, indicating some class imbalance challenges.

NNLM-50 Embeddings With NNLM-50 embeddings, the model reached a train accuracy of 87.72% a slightly lower test accuracy of 87.21%. It excelled in classifying class 1 (precision: 0.93, recall: 0.89) but underperformed on class 0 (precision: 0.74, recall: 0.82). While the training and validation metrics showed consistent improvement, the model's imbalance in class recognition suggests a need for enhanced handling of minority class patterns.

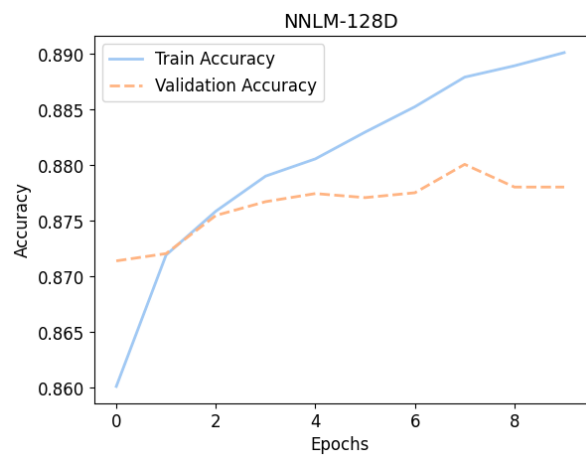
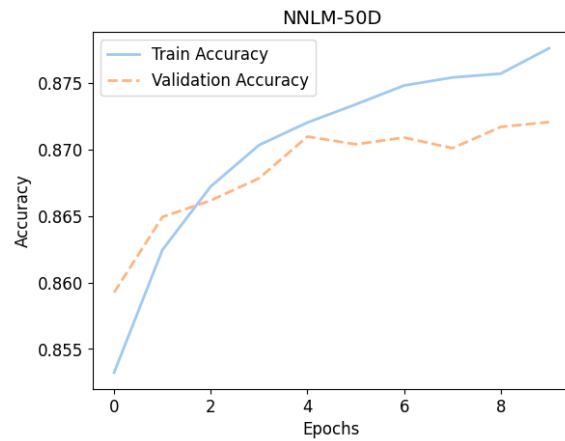
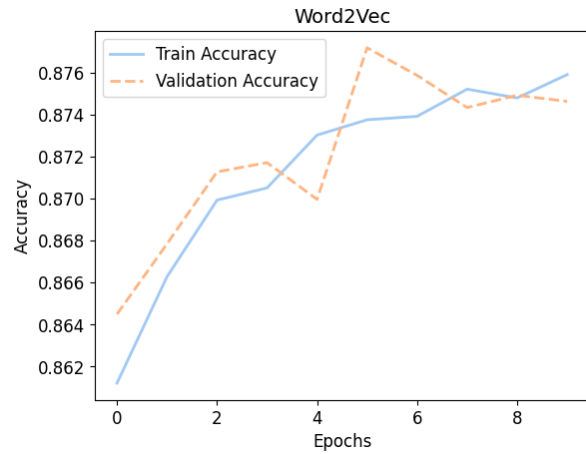
NNLM-128 Embeddings The NNLM-128 model delivered the highest train and test accuracy at 88.08% and 87.81% respectively, with well-balanced performance across both classes achieving precision and recall scores of 0.76 and 0.83 for class 0, and 0.93 and 0.90 for class 1. The model showed strong generalization with steadily decreasing loss and increasing accuracy, indicating its superior capability in capturing nuanced language patterns compared to the other embeddings.

4.2 Summary

The NNLM-128 model showed the best overall performance in classifying URLs. However, since the primary goal is to accurately predict scam URLs (class 1), there is room for improvement. The model can perform better by prioritizing recall and precision for scam detection through methods such as rebalancing the dataset, optimizing for class 1 during training, and exploring more targeted architectures or ensemble approaches. These enhancements will improve the model's effectiveness in identifying scam URLs more reliably.

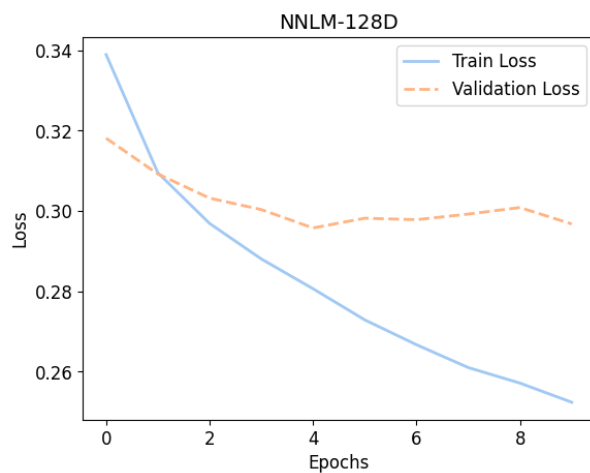
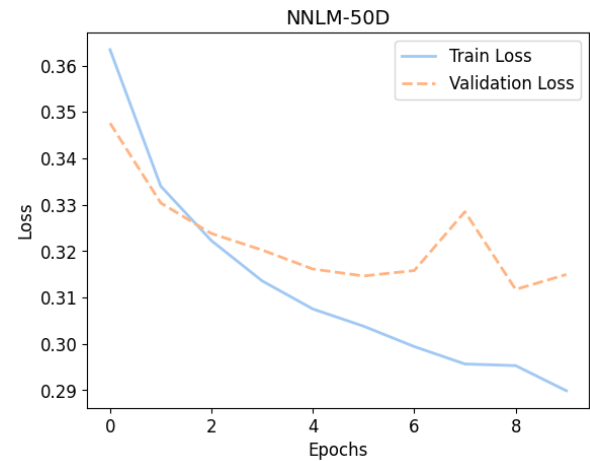
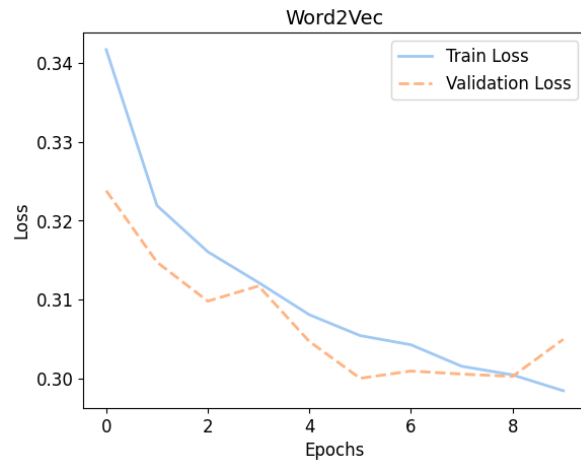
4.3 Training and Validation Accuracy

The training and validation accuracy across all techniques demonstrate consistent model performance, with each technique achieving high accuracy on the datasets. The visualizations for each technique are shown below:



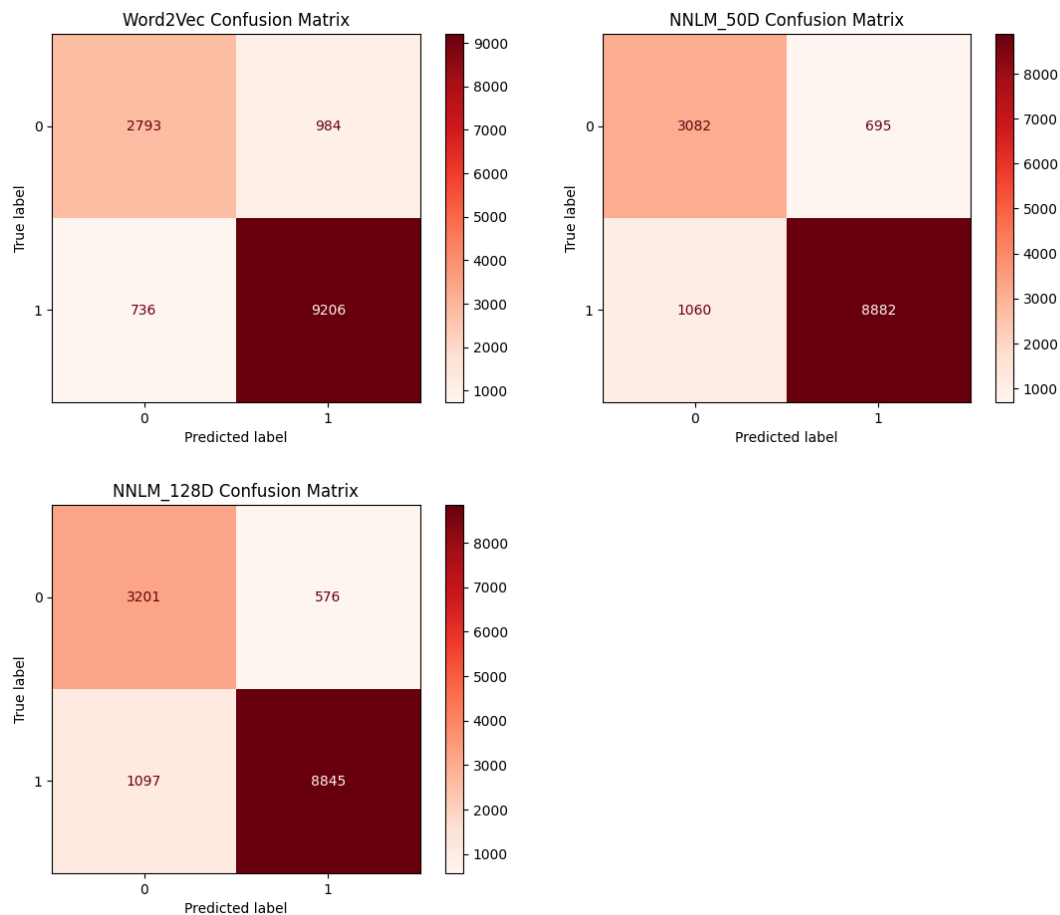
4.4 Training and Validation Loss

The training and validation loss curves across all techniques reveal key insights into model generalization. Most methods show a steady decline in both training and validation loss, indicating effective learning. The visualizations for each technique are shown below:



Confusion Matrix

The confusion matrix shows the performance of the model in terms of Positive and Negative. In this case of this task, it is 0 and 1 (legitimate and Scam). The key focus is on True Positives (TP) and True Negatives (TN), which are critical indicators of the model's ability to correctly classify scam and legitimate URLs, respectively.



Since the goal is to prevent mistakenly reporting real URLs, Word2Vec is the ideal option for maximal fraud detection. NNLM-128D is chosen in user-facing apps where user experience and trust are crucial. In comparison to Word2Vec, NNLM-50D is a dependable middle-ground option that lowers the frequency of false positives while offering respectable protection against scams.

CONCLUSION

This report provided a thorough analysis of the use of deep learning models and Natural Language Processing (NLP) approach, Text Classification for identifying scam cryptocurrency URLs. The main goal was to create a strong classification system that could accurately differentiate between legitimate and scam URLs. The study showed how semantic representations might enhance detection performance by a thorough trial with embedding techniques, including Word2Vec, NNLM-50, and NNLM-128.

Key Findings

1. Performance Comparison:

- The NNLM-128 embedding model achieved the highest test accuracy (87.81%), outperforming Word2Vec (87.46%) and NNLM-50 (87.21%).
- NNLM-128 also exhibited balanced precision and recall for both classes, indicating its superior ability to generalize across different URL structures.

2. Impact of Embeddings:

- Word2Vec performed well in detecting scam URLs (high recall for class 1) but struggled slightly with legitimate URLs (lower precision for class 0).
- NNLM-50 provided a middle-ground solution, while NNLM-128 proved most effective due to its higher-dimensional semantic representations.

3. Class Imbalance:

The dataset imbalance (18,595 legitimate vs. 50,000 scam URLs) influenced model performance, particularly in classifying legitimate URLs. Future work should explore resampling or weighted loss functions to mitigate this issue.

4. Feature Engineering:

Extracting structural and lexical features (e.g., URL length, suspicious keywords, domain age) enhanced model performance by complementing the embeddings.

Recommendations for Future Work

1. Model Enhancement:

- Experiment with transformer-based embeddings (e.g., BERT) to capture deeper contextual relationships in URLs.
- Test ensemble models (e.g., combining Word2Vec with NNLM) to improve classification robustness.

2. Dataset Improvement:

- Expand the dataset with more balanced and diverse samples to reduce bias and improve generalization.
- Include multilingual scam URLs to enhance the model's applicability in global contexts.

3. Real-World Deployment:

- Integrate the model into browser extensions or API-based security tools for real-time phishing detection.

Finally, this study successfully demonstrated that deep learning models, combined with advanced embedding techniques, can effectively identify fraudulent cryptocurrency URLs. While NNLM-128 emerged as the best-performing model, further refinements such as addressing class imbalance and exploring more sophisticated architectures could enhance detection accuracy. The findings contribute to the broader effort of combating online financial fraud, offering a scalable and automated solution for improving cybersecurity in the cryptocurrency domain.

By continuing to refine these techniques, future research can further strengthen defenses against evolving phishing strategies, ultimately safeguarding users from financial losses and maintaining trust in digital transactions.

REFERENCES

Coursera. (n.d.). Natural Language Processing Specialization. Retrieved from <https://www.coursera.org/specializations/natural-language-processing#courses>

CoinGecko. (n.d.). API documentation. Retrieved from <https://docs.coingecko.com/reference/introduction>

DeFiLlama. (n.d.). API documentation. Retrieved from <https://defillama.com/docs/api>

MetaMask. (n.d.). Phising URLs Repository. GitHub. Retrieved from <https://github.com/MetaMask/eth-phishing-detect/tree/main/src>

Seigdev. (n.d.). artificial-intelligence Repository. GitHub. Retrieved from <https://github.com/seigdev/artificial-intelligence>

Alameer, A. (n.d.). NLP Repository. GitHub. Retrieved from <https://github.com/Ali-Alameer/NLP>

Seigdev. (n.d.). Crypto Dataset Jupyter Notebook. GitHub. Retrieved from https://github.com/seigdev/artificial-intelligence/blob/main/crypto_dataset.ipynb